

Кейс 2: «Авито Очередь»

На Авито регулярно появляются товары, которые пользователи хотят купить прямо сейчас: редкие позиции, распродажи, лимитированные коллекции. В такие моменты спрос превышает доступное количество, и несколько пользователей могут одновременно перейти к покупке одного и того же товара.

При этом оплата работает асинхронно: один пользователь уже начал оформлять заказ, другой тоже дошел до оплаты. В результате количество заказов может превысить количество доступных товаров. Пользователь проходит часть пути к покупке, тратит время, но в конце может узнать, что товар уже недоступен.

Такой опыт воспринимается как несправедливый и снижает лояльность к площадке. Для бизнеса это тоже риск: растет нагрузка на систему, появляются ошибки в заказах.

В Авито подобную задачу решает сервис пользовательской очереди. Он ставит пользователя в очередь перед чекаутом, проверяет доступность товара, выдает временное право на покупку и следит, чтобы неоплаченная покупка не блокировала товар бесконечно.

В рамках кейса вам нужно создать упрощенное веб-приложение, которое показывает эту продуктовую механику.

Бизнес-задача

Создайте MVP веб-приложения, которое:

Реализует сценарий «очередь»

Пользователь должен выбрать дефицитный товар, попасть в очередь, дождаться решения системы и, если товар доступен, получить возможность перейти к покупке за ограниченное время.

Защищает от некорректного распределения товаров между несколькими покупателями

Система не должна допускать ситуации, когда несколько пользователей одновременно получают возможность купить один и тот же товар, если доступна только одна единица.

Связано с основным продуктом Авито

Очередь должна органично дополнять существующий сценарий покупки товара.

Что уже реализовано и не входит в задачу

Считаем, что следующие части продукта уже существуют и не требуют реализации в рамках кейса:

Регистрация
и авторизация
пользователя,

Полноценное
оформление заказа

Оплата

Списание стоков
после успешной
оплаты.

Что нужно реализовать

Создайте пользовательский сценарий

Пользователь открывает страницу лимитированного товара, нажимает кнопку покупки, попадает в очередь и получает экран ожидания. Система показывает его статус и объясняет, что происходит. Далее возможны несколько исходов. Продумайте их. Важно показать основной путь от выбора товара до финального состояния.

Команда сама определяет правила очереди, но они должны быть последовательными, проверяемыми и понятными.

Проработайте право на покупку

Право на покупку — центральная бизнес-механика кейса. Оно должно быть временным, персональным и защищенным.

Главное: право на покупку нельзя передать другому пользователю или использовать повторно.

Защитите сценарий от обхода

В решении нужно показать, что пользователь не может миновать очередь, если она действует для товара. Перед переходом к покупке система должна проверять, есть ли у пользователя активное право на покупку именно этого товара.

Сделайте понятные состояния для пользователя

Пользователь не должен видеть «серую зону», где непонятно, что происходит. В MVP должны быть понятные состояния - продумайте их. Каждое состояние должно содержать сообщение и следующий шаг.

Сформулируйте и опишите документацию

Опишите, какой пользовательский путь вы спроектировали и почему. Важно не просто перечислить экраны, а объяснить продуктовую логику: почему именно такие правила помогают бизнесу снизить ошибки и повысить вероятность покупки.

Продумайте и предложите дополнительные функции

Они усилят решение, если работают на бизнес-задачу. Важно: дополнительные функции не должны заменять основную механику.

Важно: нужно не просто сделать экран ожидания, а решить продуктовую задачу. Почему система не продаст один товар нескольким людям? Почему пользователь не сможет обойти очередь? Почему пользователь останется на платформе, даже если товар закончился - возможно, платформа предложит ему аналогичный лот у другого продавца?

Требования по стеку

Frontend

- React 18+
- Typescript

Разрешено использование любых внешних библиотек, в том числе:

- Дизайн-система/UI-kit (Material-UI, Ant Design)
- Стейт-менеджмент (Redux, MobX, Effector)
- Линтер (ESLint)
- Prettier
- Система сборки (Webpack, Vite)
- Библиотека для работы с асинхронными HTTP-запросами (React Query, Axios)

Backend

- Язык программирования – Golang
- База данных — выбрать самостоятельно, к выбору приложить обоснование, почему выбрана та или иная БД.
- Деплой — приложение поднимается через Docker Compose
- Описать конфигурацию линтера (.golangci.yaml в корне проекта)

Оформление решения

- | | | |
|--|---|-------------------------|
| • Общий репозиторий с frontend + backend | • Приложение поднимается через docker-compose | • Описанный README файл |
|--|---|-------------------------|

Использование ИИ

- | | |
|---|---|
| • Вы можете пользоваться нейросетями, но делайте это прозрачно. Укажите дополнительно — на каких этапах и каким образом вы ими пользовались | • Если сможете интегрировать ИИ в ваш MVP — это станет плюсом при оценке вашего решения |
|---|---|

Как жюри будет оценивать ваш результат

По каждому пункту можно получить от 0 до 5 баллов.

- **Обязательная бизнес-логика проекта**

Отличный результат: основной сценарий кейса работает полностью - пользователь может пройти путь от начала до конца, все ключевые состояния предусмотрены, логика соответствует бизнес-задаче.

- **Реализованный доп. функционал**

Отличный результат: дополнительные функции усиливают бизнес-задачу, органично выглядят в сценарии и не мешают обязательному MVP.

- **Наличие README**

Отличный результат: README позволяет запустить и проверить проект без устных пояснений команды; описаны запуск, зависимости, сценарии, API, ограничения MVP и ключевые решения.

- **Наличие тестов**

Отличный результат: покрытие юнит-тестами самых важных участков кода.

- **Наличие линтера**

Отличный результат: наличие на бэкенде и фронтенде линтеров с правилами и их обоснованием.

- **Дизайн и архитектура приложения**

Отличный результат: интерфейс понятный и аккуратный, архитектура проекта логичная, части приложения разделены по зонам ответственности.

- **Продуктовые решения, позволяющие достичь бизнес цели**

Отличный результат: понятное обоснование продуктовых решений, которыми достигается бизнес-цель.

- **Чистота и качество кода, консистентность стиля**

Отличный результат: код читаемый, структурированный, с понятными названиями, без мертвого кода и костылей. Настроены инструменты автоматической проверки качества.

- **Доступность в интернете**

Отличный результат: проект развернут и доступен по ссылке, жюри может проверить основной сценарий без локального запуска.